Binding, Bilinear Forms, and the Geometric Product

What the wedge and the geometric product do for binding, and how FGA reads the result as meaning

James Pustejovsky

Brandeis University
Department of Computer Science

CS 135: Computational Semantics

Fall 2026

Today

I	The binding problem, again	who did what to whom; the outer product; why every compression we tried forgot the order
II	The geometric product as a binding	order kept in a sign; exact unbinding; superposition, with the crosstalk in a different grade
III	Relations in two slots	a two-place predicate as a bilinear form, stored so that it can be applied one argument at a time
IV	Reading the algebra as meaning	types as grades, application as contraction, a derivation of *John loves Mary*
V	Close	back to the semester's open problems, and the next two lectures

The thread: the bilinear forms of linear algebra, the binding of VSAs, and the geometric product all start from the same object, the outer product of two vectors. They differ in what they keep of it.

Where the Semester Left Us

Session 9 ended with a ledger of what distributional and contextual vectors had and had not solved; Sessions 11 and 12 added binding.

problem	**where we met it**	**status**
word senses	*bank*, *suit*: one vector averages the senses	fixed by context
binding and order	*dog bites man* and *man bites dog* sum to the same vector	partly: HRR, MAP
composition	Session 6b: exact with tensors, at a cost that grows with the type	open
negation, antonymy	*hot* and *cold* are near neighbors	open
monotonicity	*every dog* vs. *every animal*: which inference is valid?	open

The plan for the last three lectures.

- **Today**: binding and composition: what the geometric product does for them, and how FGA reads it.
- **Next**: lexical meaning: modification, selection, coercion, regular polysemy.
- **Last**: negation, antonymy, connectives, quantifiers and monotonicity.

Part I

The Binding Problem, Again

every compression we tried forgot the order

Who Did What to Whom

sentence	agent	patient
The dog bit the man.	dog	man
The man bit the dog.	man	dog
John loves Mary.	John	Mary
Mary loves John.	Mary	John

Each pair uses the same words, and the sentences are true in different situations. Adding the word vectors gives the same vector for both members of a pair:

$$\mathbf{dog} + \mathbf{bite} + \mathbf{man} \;=\; \mathbf{man} + \mathbf{bite} + \mathbf{dog}$$

A representation of a sentence has to do three things the sum cannot:

1. record which filler is in which role: *dog* as agent, *man* as patient;
2. record the order of a relation's arguments, so that *love*(j, m) and *love*(m, j) differ;
3. let us read each filler back out, given its role.

Every binding scheme in the course is an answer to these three requirements.

The Outer Product Keeps Everything

Session 11's pair, $\mathbf{u} = (1, 2, 0, 1)$ and $\mathbf{v} = (2, 0, 1, 1)$. The outer product $M = \mathbf{u}\mathbf{v}^\top$ keeps every product $u_i v_j$:

$$\mathbf{u}\mathbf{v}^\top = \begin{pmatrix} 2 & 0 & 1 & 1 \\ 4 & 0 & 2 & 2 \\ 0 & 0 & 0 & 0 \\ 2 & 0 & 1 & 1 \end{pmatrix} \qquad \mathbf{v}\mathbf{u}^\top = \begin{pmatrix} 2 & 4 & 0 & 2 \\ 0 & 0 & 0 & 0 \\ 1 & 2 & 0 & 1 \\ 1 & 2 & 0 & 1 \end{pmatrix}$$

- **Order is kept**: $\mathbf{v}\mathbf{u}^\top$ is the transpose, a different matrix.
- **Unbinding is exact**: given $\mathbf{u}$, recover $\mathbf{v}$ as $\mathbf{v}^\top = \mathbf{u}^\top M / \|\mathbf{u}\|^2$.
- **The cost**: n^2 numbers. Here 16; at $n = 1{,}000$, a million, for one bound pair.

This is Smolensky's tensor product representation (1990). It meets all three requirements, and it is too large: a sentence with three bound pairs is three such matrices, and a nested structure is a tensor of higher order.

Two Compressions to n Numbers

Sessions 11 and 12 compressed the outer product back to n numbers in two ways:

binding	**what it takes from $\mathbf{u}\mathbf{v}^\top$**	$\mathbf{u}$ with $\mathbf{v}$	$\mathbf{v}$ with $\mathbf{u}$
MAP: $\mathbf{u} \odot \mathbf{v}$	the diagonal: $(u_1 v_1, \ldots, u_n v_n)$	$(2, 0, 0, 1)$	$(2, 0, 0, 1)$
HRR: $\mathbf{u} \circledast \mathbf{v}$	the fold: entry k sums the cells with $i + j \equiv k \pmod{n}$	$(4, 5, 2, 5)$	$(4, 5, 2, 5)$

Both are **commutative**: binding $\mathbf{u}$ to $\mathbf{v}$ and $\mathbf{v}$ to $\mathbf{u}$ give the same vector. The operation we chose in order to keep order does not keep it.

How the course worked around it

Stipulate role vectors: $\mathbf{agent} \circledast \mathbf{dog} + \mathbf{patient} \circledast \mathbf{man}$. Order is then carried by *which role vector* a filler is bound to, not by the binding itself. Unbinding is approximate in HRR and exact for a single pair in MAP, and both need a cleanup memory once several pairs are added together.

Split the Outer Product First

Every square matrix is a symmetric part plus an antisymmetric part. For $M = \mathbf{u}\mathbf{v}^\top$:

$$S = \tfrac{1}{2}\big(M + M^\top\big) = \begin{pmatrix} 2 & 2 & \frac{1}{2} & \frac{3}{2} \\ 2 & 0 & 1 & 1 \\ \frac{1}{2} & 1 & 0 & \frac{1}{2} \\ \frac{3}{2} & 1 & \frac{1}{2} & 1 \end{pmatrix} \qquad A = \tfrac{1}{2}\big(M - M^\top\big) = \begin{pmatrix} 0 & -2 & \frac{1}{2} & -\frac{1}{2} \\ 2 & 0 & 1 & 1 \\ -\frac{1}{2} & -1 & 0 & -\frac{1}{2} \\ \frac{1}{2} & -1 & \frac{1}{2} & 0 \end{pmatrix}$$

- Swapping $\mathbf{u}$ and $\mathbf{v}$ transposes M, which leaves S alone and flips the sign of A.
- So A **is where the order lives**. S is the same for *dog bites man* and *man bites dog*.
- The diagonal of A is zero, since $A_{ii} = -A_{ii}$.

This is the decomposition from the bilinear-forms sessions, applied to the outer product of two vectors instead of to a learned matrix.

Why Both Compressions Forget the Order

- **MAP** takes the diagonal of M. The diagonal of A is zero, so MAP reads only S.
- **HRR** sums cells (i, j) in groups by $i + j$. Transposing moves (i, j) to (j, i), in the same group. So each group sum of A equals its own negative, and is 0. HRR reads only S.

The general fact

Call a binding *linear in the outer product* if each output number is a fixed weighted sum of the cells $u_i v_j$. MAP, HRR, the dot product, and the tensor product all are. For such a binding ϕ,

$$\phi(\mathbf{v}\mathbf{u}^\top) = \phi(S - A) = \phi(S) - \phi(A), \qquad \phi(\mathbf{u}\mathbf{v}^\top) = \phi(S) + \phi(A).$$

So ϕ is commutative exactly when $\phi(A) = 0$: **a commutative binding is one that ignores the antisymmetric half**.

To keep order, a binding must keep some of A. The question is how much of A, and of S, to keep, and in what form.

Bilinear Forms Read the Same Matrix

A bilinear form, $\mathbf{x}^\top G\,\mathbf{y}$, is a binding that returns one number. Written with the outer product:

$$\mathbf{x}^\top G\,\mathbf{y} \;=\; \sum_{i,j} G_{ij}\, x_i y_j \;=\; \sum_{i,j} G_{ij}\,\left(\mathbf{x}\mathbf{y}^\top\right)_{ij}$$

A weighted reading of the outer product, with the weights in G. Split both G and $\mathbf{x}\mathbf{y}^\top$, and the cross terms vanish:

- the symmetric part of G reads only the symmetric half of $\mathbf{x}\mathbf{y}^\top$;
- the antisymmetric part of G reads only the antisymmetric half.

(A symmetric matrix weighted against an antisymmetric one sums to zero, by the same argument as the fold.)

The attention head, again

A head's score $\mathbf{x}_i^\top G\,\mathbf{x}_j$ reads the outer product of query and key. Its symmetric part scores how alike they are; its antisymmetric part scores their order. With the head from the bilinear-forms session, $G = \left(\begin{smallmatrix} 1 & 2 \\ 0 & 1 \end{smallmatrix}\right)$, query $(3, 1)$ and key $(1, 3)$: $24 = 16 + 8$, and reversed, $8 = 16 - 8$.

Bindings, Sorted by What They Keep

For Session 11's $\mathbf{u}$ and $\mathbf{v}$, with $n = 4$:

operation	what it keeps of $\mathbf{u}\mathbf{v}^\top$	numbers	order?	course
tensor product	all of it	$n^2 = 16$	yes	Sessions 6b, 11
MAP	the diagonal, in S	$n = 4$	no	Session 12
HRR	the fold of S	$n = 4$	no	Session 11
dot product	the trace of S	1	no	throughout
wedge	all of A	$\binom{n}{2} = 6$	yes, as a sign	Session 13a
geometric product	the trace of S, and all of A	$1 + 6 = 7$	yes	Session 13a

The geometric product makes a different choice from the VSAs. MAP and HRR keep n numbers of S and none of A. The geometric product keeps one number of S, the dot product, and *all* of A.

Part II asks whether that choice meets the three requirements: roles, order, and reading back.

Part II

The Geometric Product as a Binding

order in a sign, exact unbinding, crosstalk in another grade

The Geometric Product, Restated

From Sessions 13a and 13b: the product of basis vectors obeys two rewrite rules,

$$\mathbf{e}_i\mathbf{e}_i = 1, \qquad \mathbf{e}_i\mathbf{e}_j = -\,\mathbf{e}_j\mathbf{e}_i \quad (i \neq j)$$

and we write $\mathbf{e}_{ij}$ for $\mathbf{e}_i\mathbf{e}_j$ with $i < j$: the unit plane spanned by $\mathbf{e}_i$ and $\mathbf{e}_j$, oriented from $\mathbf{e}_i$ toward $\mathbf{e}_j$. Multiplying out two vectors with these rules gives

$$\mathbf{uv} \;=\; \underbrace{\sum_i u_i v_i}_{\mathbf{u}\cdot\mathbf{v}} \;+\; \underbrace{\sum_{i<j} \left(u_i v_j - u_j v_i\right) \mathbf{e}_{ij}}_{\mathbf{u}\wedge\mathbf{v}}$$

- The terms with $i = j$ give the dot product, by the first rule.
- The terms $u_i v_j\, \mathbf{e}_i\mathbf{e}_j$ and $u_j v_i\, \mathbf{e}_j\mathbf{e}_i$ meet in one plane, with opposite signs, by the second rule. Each plane gets a signed area.

Grade counts the distinct basis vectors in a term: 1 has grade 0, $\mathbf{e}_i$ grade 1, $\mathbf{e}_{ij}$ grade 2, $\mathbf{e}_{ijk}$ grade 3. The product of two vectors has a grade-0 part and a grade-2 part.

Session 11's Pair as a Geometric Product

With $\mathbf{u} = (1, 2, 0, 1)$ and $\mathbf{v} = (2, 0, 1, 1)$, there are six planes in four dimensions:

plane	$\mathbf{e}_{12}$	$\mathbf{e}_{13}$	$\mathbf{e}_{14}$	$\mathbf{e}_{23}$	$\mathbf{e}_{24}$	$\mathbf{e}_{34}$
$u_i v_j - u_j v_i$	-4	1	-1	2	2	-1
$2A_{ij}$ from the split	-4	1	-1	2	2	-1

$$\mathbf{uv} \;=\; 3 \;-\; 4\,\mathbf{e}_{12} + \mathbf{e}_{13} - \mathbf{e}_{14} + 2\,\mathbf{e}_{23} + 2\,\mathbf{e}_{24} - \mathbf{e}_{34}, \qquad \mathbf{vu} \;=\; 3 \;+\; 4\,\mathbf{e}_{12} - \cdots$$

- The scalar part 3 is the trace of S: the dot product.
- The bivector part holds exactly the entries of A above the diagonal, doubled. Nothing of A is lost.
- Swapping the factors flips the whole bivector part and leaves the scalar: **the order is in the sign**.

In the language of Session 13a: the dot product measures how much $\mathbf{v}$ lies along $\mathbf{u}$; the wedge records, plane by plane, how $\mathbf{v}$ turns away from $\mathbf{u}$.

A Unit Vector Undoes Itself

Multiply a vector by itself. The wedge of a vector with itself is zero, since every $u_i u_j - u_j u_i = 0$, so

$$\mathbf{aa} \;=\; \mathbf{a}\cdot\mathbf{a} \;=\; \|\mathbf{a}\|^2, \qquad \text{and for a unit vector,} \qquad \mathbf{aa} = 1.$$

The geometric product is associative, so a role undoes its own binding:

$$\mathbf{r}\,(\mathbf{rf}) \;=\; (\mathbf{rr})\,\mathbf{f} \;=\; \mathbf{f}$$

exactly, for any filler $\mathbf{f}$, with no assumption about $\mathbf{f}$.

	inverse of a role	**one bound pair**
HRR	the involution $\mathbf{r}^*$: approximate	recovered with noise
MAP	$\mathbf{r}$ itself, since $\mathbf{r} \odot \mathbf{r} = \mathbf{1}$ for ± 1 vectors	recovered exactly
geometric product	$\mathbf{r}$ itself, since $\mathbf{rr} = 1$ for unit vectors	recovered exactly

The geometric product shares MAP's best property, a role that is its own inverse, and adds the one MAP lacks: $\mathbf{rf} \neq \mathbf{fr}$.

Unbinding, Worked in the Plane

Take $\mathbf{u} = \mathbf{e}_1 + 2\mathbf{e}_2$ and $\mathbf{v} = 3\mathbf{e}_1 + \mathbf{e}_2$. Bind:

$$\mathbf{uv} = (1 \cdot 3 + 2 \cdot 1) + (1 \cdot 1 - 2 \cdot 3)\,\mathbf{e}_{12} = 5 - 5\,\mathbf{e}_{12}, \qquad \mathbf{vu} = 5 + 5\,\mathbf{e}_{12}$$

$\mathbf{u}$ is not a unit vector, so its inverse is $\mathbf{u}/\|\mathbf{u}\|^2 = 0.2\,\mathbf{e}_1 + 0.4\,\mathbf{e}_2$. Unbind, using $\mathbf{e}_1\mathbf{e}_{12} = \mathbf{e}_2$ and $\mathbf{e}_2\mathbf{e}_{12} = -\mathbf{e}_1$:

$$\begin{aligned} \mathbf{u}^{-1}(\mathbf{uv}) &= (0.2\,\mathbf{e}_1 + 0.4\,\mathbf{e}_2)(5 - 5\,\mathbf{e}_{12}) \\ &= \mathbf{e}_1 + 2\,\mathbf{e}_2 - \mathbf{e}_1\mathbf{e}_{12} - 2\,\mathbf{e}_2\mathbf{e}_{12} \\ &= \mathbf{e}_1 + 2\,\mathbf{e}_2 - \mathbf{e}_2 + 2\,\mathbf{e}_1 \;=\; 3\,\mathbf{e}_1 + \mathbf{e}_2 \;=\; \mathbf{v} \end{aligned}$$

Exactly $\mathbf{v}$. Compare HRR on Session 11's pair: binding gives $(4, 5, 2, 5)$, and unbinding with the involution of $\mathbf{u}$ gives $(19, 13, 17, 15)$, at cosine 0.88 with $\mathbf{v}$. Close, but it needs a cleanup memory to become $\mathbf{v}$.

Superposing Several Bindings

A sentence binds several fillers to several roles and adds the results:

$$\mathbf{E} \;=\; \mathbf{r}_1\mathbf{f}_1 + \mathbf{r}_2\mathbf{f}_2$$

To read the first filler, multiply by its role:

$$\mathbf{r}_1\mathbf{E} \;=\; \underbrace{\mathbf{r}_1\mathbf{r}_1\mathbf{f}_1}_{\mathbf{f}_1} \;+\; \underbrace{\mathbf{r}_1\mathbf{r}_2\mathbf{f}_2}_{\text{crosstalk}}$$

The first term is exact. The second is the other binding, seen through the wrong role. Every binding scheme has this term; they differ in where it ends up.

scheme	**the crosstalk term**
MAP	$\mathbf{r}_1 \odot \mathbf{r}_2 \odot \mathbf{f}_2$: a random-looking ± 1 vector, in the same space as $\mathbf{f}_1$. It cannot be told apart from a filler, so a cleanup memory compares the result with every known filler. In Session 12, a three-pair record returned its fillers at cosine 0.50.
geometric product	$\mathbf{r}_1\mathbf{r}_2\mathbf{f}_2$: a product of three vectors. What grade is it?

The Crosstalk Lands in Another Grade

Suppose the roles are orthogonal to each other and to the fillers. Then every product in $\mathbf{r}_1\mathbf{r}_2\mathbf{f}_2$ is between perpendicular vectors, no dot product survives, and

$$\mathbf{r}_1\mathbf{r}_2\mathbf{f}_2 \;=\; \mathbf{r}_1 \wedge \mathbf{r}_2 \wedge \mathbf{f}_2 \qquad \text{a grade-3 object}$$

Worked. Roles $\mathbf{r}_1 = \mathbf{e}_1$, $\mathbf{r}_2 = \mathbf{e}_2$; fillers $\mathbf{f}_1 = \mathbf{e}_3$, $\mathbf{f}_2 = 0.6\,\mathbf{e}_3 + 0.8\,\mathbf{e}_4$. Then $\mathbf{E} = \mathbf{e}_{13} + 0.6\,\mathbf{e}_{23} + 0.8\,\mathbf{e}_{24}$, and

	full product	**grade-1 part**	**filler**
$\mathbf{r}_1\mathbf{E}$	$\mathbf{e}_3 + 0.6\,\mathbf{e}_{123} + 0.8\,\mathbf{e}_{124}$	$\mathbf{e}_3$	$\mathbf{f}_1$, exactly
$\mathbf{r}_2\mathbf{E}$	$0.6\,\mathbf{e}_3 + 0.8\,\mathbf{e}_4 - \mathbf{e}_{123}$	$0.6\,\mathbf{e}_3 + 0.8\,\mathbf{e}_4$	$\mathbf{f}_2$, exactly

Grade projection does the work of the cleanup memory

The crosstalk is still there, but it has a different *type*: grade 3, where every filler is grade 1. Keeping the grade-1 part removes it exactly. MAP needs a memory of all fillers to do this; the algebra needs only to know which grade a filler has.

The Same Step, Named: Contraction

"Multiply by the role, then keep the grade-1 part" is common enough to have a name, the **left contraction**, written $\mathbf{r} \rfloor B$. For a role and a bound pair it has a simple formula:

$$\mathbf{r} \rfloor (\mathbf{a} \wedge \mathbf{b}) \;=\; (\mathbf{r} \cdot \mathbf{a})\,\mathbf{b} \;-\; (\mathbf{r} \cdot \mathbf{b})\,\mathbf{a}$$

Dot the role against each factor, and keep the *other* factor, with a sign.

- **Check**: $\mathbf{e}_1 \rfloor (\mathbf{e}_1 \wedge \mathbf{e}_3) = (\mathbf{e}_1 \cdot \mathbf{e}_1)\,\mathbf{e}_3 - (\mathbf{e}_1 \cdot \mathbf{e}_3)\,\mathbf{e}_1 = \mathbf{e}_3$. The role finds itself and returns its partner.
- **Grade**: a vector contracted into a grade-2 object gives grade 1. Contraction lowers grade by one. This will be function application in Part IV.
- **Cost**: the formula computes the grade-1 part directly. The grade-3 crosstalk is never built, so it costs no memory.
- **When a role is not orthogonal to its own filler**, $\mathbf{rf}$ has a scalar part as well. The product $\mathbf{r}(\mathbf{rf}) = \mathbf{f}$ is still exact; the contraction of the wedge alone gives the part of $\mathbf{f}$ perpendicular to $\mathbf{r}$.

Who Did What to Whom, Decoded

Roles in a subspace of their own: agent $= \mathbf{e}_1$, patient $= \mathbf{e}_2$. Entities in another: john $= \mathbf{e}_3$, mary $= 0.6\,\mathbf{e}_3 + 0.8\,\mathbf{e}_4$, so $\cos(\text{john}, \text{mary}) = 0.6$.

$$\textit{John loves Mary}: \quad \mathbf{E} = \mathbf{e}_1 \wedge \text{john} + \mathbf{e}_2 \wedge \text{mary} = \mathbf{e}_{13} + 0.6\,\mathbf{e}_{23} + 0.8\,\mathbf{e}_{24}$$

$$\textit{Mary loves John}: \quad \mathbf{E}' = \mathbf{e}_1 \wedge \text{mary} + \mathbf{e}_2 \wedge \text{john} = 0.6\,\mathbf{e}_{13} + 0.8\,\mathbf{e}_{14} + \mathbf{e}_{23}$$

query	**on $\mathbf{E}$**	**on $\mathbf{E}'$**
agent: $\mathbf{e}_1 \rfloor (\cdot)$	john, exactly	mary, exactly
patient: $\mathbf{e}_2 \rfloor (\cdot)$	mary, exactly	john, exactly

The two sentences have cosine 0.6, which is the similarity of John and Mary and nothing more. All three requirements of Part I are met: roles recorded, order recorded, fillers read back, with no cleanup step.

The same works for three roles: *Mary gave John a book* is agent $\wedge$ mary $+$ recipient $\wedge$ john $+$ theme $\wedge$ book, and each role returns its filler exactly.

What the Grades Cost

Exactness was bought with room. The number of independent directions at each grade, in n dimensions:

	grade 1: fillers	**grade 2: bound pairs**	**grade 3: crosstalk**
count	n	$\binom{n}{2}$	$\binom{n}{3}$
$n = 4$	4	6	4
$n = 1{,}000$	$1{,}000$	$499{,}500$	about 1.7×10^8

- A record of bound pairs lives in grade 2: about $n^2/2$ numbers, like the antisymmetric half of a tensor. That is the price of keeping A, and Session 11 showed there is no cheaper place to put it.
- The grade-3 crosstalk is never stored, since contraction computes only the grade-1 part.
- The exactness needs the roles orthogonal to each other and to the fillers. With learned vectors that hold only approximately, some crosstalk comes back into grade 1; how much, against HRR at the same size, is an empirical question.

Two Ways to Bind

	HRR and MAP	**the geometric product**
what is kept of $\mathbf{u}\mathbf{v}^\top$	n numbers of S	the trace of S, and all of A
order	by role vectors; the binding commutes	in the sign, and by role vectors
a bound pair is	a vector, like a filler	grade 2: it says what it is
unbinding one pair	approximate (HRR) or exact (MAP)	exact
superposition crosstalk	same type as a filler: cleanup memory	a different grade: projection
size	fixed: n	grows with grade

Neither column is better everywhere. A VSA keeps everything in one space of fixed size, which suits memory and retrieval with many items. The geometric product keeps structure in its type, which suits argument structure, where the number of roles is small.

FGA takes the second column, and then asks what the grades mean.

Part III
Relations in Two Slots

a bilinear form, stored so it can be applied one argument at a time

From Roles to Relations

Part II recorded *who* filled each role. A verb such as *love* has a second job: it must say, for any pair of entities, how well the relation holds. Linear algebra's answer, from DisCoCat and from attention, is a bilinear form.

Take a two-dimensional entity space with john $= (1, 0)$ and mary $= (0.6, 0.8)$, the same cosine 0.6 as before, and

$$G_{\text{love}} = \begin{pmatrix} 0.2 & 0.9 \\ 0.3 & 0.1 \end{pmatrix}, \qquad \text{love}(\mathbf{x}, \mathbf{y}) = \mathbf{x}^\top G\, \mathbf{y} \quad \text{(row: subject; column: object)}$$

sentence	**score**	**computed**
John loves Mary	0.84	$(1,0)\,G\,(0.6,0.8)^\top = 0.2(0.6) + 0.9(0.8)$
Mary loves John	0.36	$(0.6,0.8)\,G\,(1,0)^\top$
John loves himself	0.20	$(1,0)\,G\,(1,0)^\top$
Mary loves herself	0.71	$(0.6,0.8)\,G\,(0.6,0.8)^\top$

Four numbers determine the relation on every pair. In n dimensions, n^2 numbers.

The Split of a Relation, Read Linguistically

$$G = \underbrace{\begin{pmatrix} 0.2 & 0.6 \\ 0.6 & 0.1 \end{pmatrix}}_{S} + \underbrace{\begin{pmatrix} 0 & 0.3 \\ -0.3 & 0 \end{pmatrix}}_{A}$$

$$\textit{John loves Mary} = 0.60 + 0.24 = 0.84$$
$$\textit{Mary loves John} = 0.60 - 0.24 = 0.36$$

construction	**reads**	**why**
symmetric predicates: *marry*, *meet*, *resemble*	S only: $A = 0$	*John married Mary* entails *Mary married John*
reciprocal: *John and Mary love each other*	S: $2\,\mathbf{j}^\top S\,\mathbf{m} = 1.2$	both directions, summed: A cancels
reflexive: *John loves himself*	S on the diagonal: 0.2	$\mathbf{x}^\top A\,\mathbf{x} = 0$ for every $\mathbf{x}$
order-sensitive: *precede*, *be taller than*	mainly A	reversing the arguments reverses the verdict

The split tracks distinctions that grammars mark: reciprocals and reflexives are constructions about the symmetric part of a relation, and symmetric predicates are relations with no antisymmetric part.

One Entity Space Is Not Enough

Part II stored bindings as wedges. Try storing *love* as a bivector over the entity space itself. A wedge in one space can hold only the antisymmetric part, so the best it can do is $0.3\,\mathbf{e}_1 \wedge \mathbf{e}_2$, which is A:

sentence	with G	with a bivector over entities
John loves Mary	0.84	0.24
Mary loves John	0.36	-0.24
John loves himself	0.20	0
John and Mary love each other	1.20	0

Reflexives and reciprocals vanish, because $\mathbf{x} \wedge \mathbf{x} = 0$. And a symmetric predicate such as *marry*, with $A = 0$, vanishes entirely.

What went wrong

The subject and the object were vectors in the same space, so the wedge treated them as interchangeable up to a sign. A relation needs its two argument positions to be *different places*, so that john as subject and john as object are not the same vector.

Two Slots: A Subject Copy and an Object Copy

Give each argument position its own copy of the entity space, all four directions mutually orthogonal:

$$\text{subject slot: } \mathbf{s}_1, \mathbf{s}_2 \qquad\qquad \text{object slot: } \mathbf{o}_1, \mathbf{o}_2$$

An entity enters a slot with the same coordinates: John as subject is $\mathbf{s}_1$; Mary as object is $0.6\,\mathbf{o}_1 + 0.8\,\mathbf{o}_2$. Now store the relation as a sum of wedges *across* the slots, with G's entries as weights:

$$L_{\text{love}} \;=\; \sum_{i,k} G_{ik}\; \mathbf{s}_i \wedge \mathbf{o}_k \;=\; 0.2\,\mathbf{s}_1 \wedge \mathbf{o}_1 + 0.9\,\mathbf{s}_1 \wedge \mathbf{o}_2 + 0.3\,\mathbf{s}_2 \wedge \mathbf{o}_1 + 0.1\,\mathbf{s}_2 \wedge \mathbf{o}_2$$

- Four numbers, the same four as G: nothing is lost, symmetric or antisymmetric.
- No two terms cancel. $\mathbf{s}_1 \wedge \mathbf{o}_2$ and $\mathbf{s}_2 \wedge \mathbf{o}_1$ are different planes, so the orientation that the single space folded together is kept apart.
- L is a grade-2 object, like a record of bound pairs: a relation is *a bound pair of slots*, weighted.

Applying the Relation, One Argument at a Time

Contracting from the right works like contracting from the left: $(\mathbf{a} \wedge \mathbf{b}) \lfloor \mathbf{c} = (\mathbf{b} \cdot \mathbf{c})\,\mathbf{a} - (\mathbf{a} \cdot \mathbf{c})\,\mathbf{b}$. Supply the object, mary in the object slot:

$$\begin{aligned} L \lfloor (0.6\,\mathbf{o}_1 + 0.8\,\mathbf{o}_2) &= \big(0.2(0.6) + 0.9(0.8)\big)\,\mathbf{s}_1 + \big(0.3(0.6) + 0.1(0.8)\big)\,\mathbf{s}_2 \\ &= 0.84\,\mathbf{s}_1 + 0.26\,\mathbf{s}_2 \end{aligned}$$

A vector in the subject slot: the property *loves Mary*. Every term with an object direction was used up; the subject directions remain. Now supply a subject:

subject	**contraction with** $0.84\,\mathbf{s}_1 + 0.26\,\mathbf{s}_2$	**sentence**
john $= \mathbf{s}_1$	0.84	*John loves Mary*
mary $= 0.6\,\mathbf{s}_1 + 0.8\,\mathbf{s}_2$	$0.6(0.84) + 0.8(0.26) = 0.71$	*Mary loves herself*

The same numbers as $\mathbf{x}^\top G\,\mathbf{y}$, as they must be: two contractions of L compute the bilinear form, one argument at a time.

Active and Passive

The arguments can also arrive in the other order. Supply the subject first:

$$\mathbf{s}_1 \rfloor L \;=\; 0.2\,\mathbf{o}_1 + 0.9\,\mathbf{o}_2 \qquad \text{the property } \textit{loved by John}\text{, in the object slot}$$

$$(0.2\,\mathbf{o}_1 + 0.9\,\mathbf{o}_2) \lfloor (0.6\,\mathbf{o}_1 + 0.8\,\mathbf{o}_2) \;=\; 0.12 + 0.72 \;=\; 0.84$$

sentence	**supplied first**	**result**
John loves Mary.	John, into the subject slot	0.84
Mary is loved by John.	Mary, into the object slot	0.84

The slot carries the role, not the position in the string. Active and passive supply the arguments in different orders, and since the slots are orthogonal, the order of contraction does not change the result. In Session 12b, a toy attention head needed a second row of weights to handle the passive; here the relation needs nothing new.

What the algebra does not decide: which slot an argument goes to. That is still syntax's job: something must put mary in the object slot of a passive.

What Two Slots Keep, and What They Cost

	a symmetric form	**a bivector over one entity space**	**a bivector across two slots**
holds	S	A	all of G
numbers	$\frac{1}{2}n(n+1)$	$\binom{n}{2}$	n^2
reflexive, reciprocal	yes	lost	yes
order of arguments	lost	yes	yes
partial application	a vector	a vector	a vector in the right slot

Compared with the matrix G: the same n^2 numbers, so no saving. What the two-slot form adds is *type*. L is grade 2; each argument lowers the grade by one; and a partial application, such as *loves Mary*, is a grade-1 vector in a known slot, ready for the next argument.

This is Session 6b's tensor semantics, where a two-place predicate was an order-2 tensor, rewritten so that the order of a tensor becomes the grade of one object in one algebra.

Part IV

Reading the Algebra as Meaning

FGA: types as grades, application as contraction

What FGA Adds to the Algebra

Parts II and III used the geometric product as a tool. **Functional Geometric Algebra** (FGA) makes a semantic claim about it: the grades and operations of the algebra correspond to the types and operations of a compositional semantics.

in formal semantics	**in FGA**	**seen today**
a type	a grade, and the subspace it lives in	entities grade 1; *love* grade 2
function application	contraction	*loves Mary* from *love*
a saturated sentence	grade 0: a degree	*John loves Mary* $= 0.84$
an argument position	a slot: an orthogonal copy of a space	subject and object slots
binding a role to a filler	the wedge, or the product	agent $\wedge$ john

The claim is a hypothesis about how meaning can be represented, not a finding about how any model does represent it. Its value has to be shown by what it explains and by experiments.

Types as Grades

Session 6b mapped semantic types to tensors of increasing order. FGA maps them to grades of one algebra. Grade alone cannot tell an entity from a property, both grade 1, so FGA also says *where* an object lives:

type	**example**	**Session 6b**	**FGA**
t	a truth value	order 0	grade 0: a scalar, a degree
e	John	order 1	grade 1, in the entity space
$\langle e, t\rangle$	laughs	order 1	grade 1, a property
$\langle e, \langle e, t\rangle\rangle$	loves	order 2	grade 2, across two slots
three-place	gives	order 3	grade 3, across three slots

For a predicate, grade counts the arguments still to come. A relation with k open positions has grade k; each argument lowers it by one; a sentence, with none left, has grade 0. An entity has grade 1 because it fills one position.

The FGA position paper also separates entity, property, and kind subspaces, so that a countable noun such as dog differs in type from a verb such as laughs.

Application Is Contraction

In Session 6b, a property applied to an entity was an inner product: laughs(john) $= \mathbf{p} \cdot \mathbf{x}$. For two vectors, contraction *is* the inner product, so FGA keeps that result unchanged:

$$\mathbf{p} \rfloor \mathbf{x} \;=\; \mathbf{p} \cdot \mathbf{x}$$

With laughs $= (0.9, 0.1)$ in the entity space:

sentence	**degree**	**computed**
John laughs	0.90	$(0.9, 0.1) \cdot (1, 0)$
Mary laughs	0.62	$(0.9, 0.1) \cdot (0.6, 0.8) = 0.54 + 0.08$

What contraction adds is the step from one argument to several. For a relation, contraction takes one argument and returns an object of lower grade, still waiting for the rest. That is currying, the way Session 6b's λ-terms took their arguments one at a time:

$$\lambda y.\, \lambda x.\, love(x, y) \qquad \longleftrightarrow \qquad L_{\mathsf{love}},\ \text{contracted by the object, then by the subject}$$

A Derivation: *John Loves Mary*

expression	**object**	**grade**	**in λ terms**
loves	$L = 0.2\,\mathbf{s}_1 \wedge \mathbf{o}_1 + 0.9\,\mathbf{s}_1 \wedge \mathbf{o}_2 + 0.3\,\mathbf{s}_2 \wedge \mathbf{o}_1 + 0.1\,\mathbf{s}_2 \wedge \mathbf{o}_2$	2	$\lambda y.\, \lambda x.\, love(x, y)$
Mary, as object	$0.6\,\mathbf{o}_1 + 0.8\,\mathbf{o}_2$	1	m
loves Mary	$L \lfloor \text{mary} = 0.84\,\mathbf{s}_1 + 0.26\,\mathbf{s}_2$	1	$\lambda x.\, love(x, m)$
John, as subject	$\mathbf{s}_1$	1	j
John loves Mary	$\mathbf{s}_1 \rfloor (0.84\,\mathbf{s}_1 + 0.26\,\mathbf{s}_2) = 0.84$	0	$love(j, m)$

- **Each step is one contraction**, and each lowers the grade by one, from 2 to 1 to 0.
- **The sentence is a degree**, 0.84, not *true* or *false*. A threshold, or a comparison with alternatives, turns it into a verdict when one is needed.
- **The intermediate object is meaningful**: *loves Mary* is a property, a grade-1 vector, and can be applied to any subject.

Type Errors Are Grade Errors

In Session 6b's typed λ-calculus, some combinations were ruled out: an entity cannot apply to an entity. Grades rule out the same combinations, by returning nothing:

combination	**contraction**	**result**
loves Mary, then *John*	grade 1 into grade 1	a degree: well typed
John loves Mary, then *Bill*	a vector into a scalar	0: nothing left to fill
loves Mary, then *John* entered as an object	a subject-slot vector against an object-slot vector	0: wrong slot
loves applied to *laughs*	a property into a slot for entities	0 if properties and entities are kept in orthogonal subspaces

Grade is visible in the object, read off by projecting to each grade. In Session 12c, HRR could not tell a filler from a bound pair, since both were vectors of the same width. Here a bound pair, a relation, and a degree each have their own grade.

A contraction that returns 0 signals a mismatch; it does not explain it. Whether a model's errors look like this is one of the experimental questions.

One Product, Several Semantic Jobs

The geometric product of two vectors has a scalar part and a bivector part. FGA uses each part, and their combination, for a different semantic job:

grade	**semantic job**	**example**
0	measuring: a degree, a similarity, a truth value	*John loves Mary* $= 0.84$
1	entities and properties	john; *loves Mary*
2	bound pairs and relations	agent $\wedge$ john; L_{love}
$0 + 2$	an action: a turn in a plane (13b: a rotor)	next lecture: modification and coercion as programs that change a meaning

Session 13a's slogan, in semantic terms: the dot product *measures*, as application measures how well a property holds of an entity; the geometric product *acts*, as binding and modification change what a meaning is.

What a Transformer Keeps of the Same Product

An attention head scores a query against a key with $\mathbf{x}^\top G\,\mathbf{y}$. Measuring with the head's symmetric part S, the geometric product of query and key has a scalar part $\mathbf{x}^\top S\,\mathbf{y}$ and a bivector part, and the antisymmetric part A reads the bivector part. With the head from the bilinear-forms session, query $(3, 1)$, key $(1, 3)$:

	scalar part	**bivector part**	**the head's score**
query $(3, 1)$, key $(1, 3)$	16	$+8\,\mathbf{e}_{12}$	24
query $(1, 3)$, key $(3, 1)$	16	$-8\,\mathbf{e}_{12}$	8

The head adds the two parts into one number. After that it cannot tell "very alike, slightly out of order" from "less alike, in order". FGA keeps them as separate grades, so later operations can use either.

Whether a model gains anything by keeping the parts apart is an empirical question. The extra lecture on sparse autoencoders works through this reading of attention heads in detail.

What the Algebra Does Not Supply

FGA fixes the operations. Several things still have to come from somewhere else:

what	**why the algebra cannot supply it**	**where it comes from**
the lexical vectors	the algebra says how john combines, not what john is	data: distributional learning
the relation's weights	L_{love} has n^2 numbers to fill	data, or a lexicon
which slot an argument fills	active and passive need a mapping from positions to slots	syntax: a parser, or a learned assignment
orthogonal roles and slots	exact unbinding assumes it	a design choice, approximated by learned vectors

Session 12b asked *who decided that Mary is the agent?* The answer is the same here: FGA gives a role a place to live, a slot, and exact operations once it is there. Getting it there is a learning or parsing problem.

Part V
Close

back to the ledger

Back to the Ledger

problem	earlier answer	today	next
binding and order	HRR, MAP: commutative binding, order from role vectors, cleanup	the product keeps all of A: order in a sign; exact unbinding; crosstalk in grade 3	
relations	a matrix G, or a tensor	a grade-2 object across two slots: same numbers, typed	
reflexives, reciprocals	not addressed	read the symmetric part of the relation	
composition	Session 6b: exact, with tensors	contraction lowers grade; partial applications are vectors	lecture 2
modification, coercion, polysemy	contextual vectors		lecture 2
negation, antonymy	open since Session 9		lecture 3
quantifiers, monotonicity	Session 6b, in tensors; open for vectors		lecture 3

Today closed the first rows, under stated assumptions: orthogonal roles and slots, and a parser that puts arguments in the right slots.

The Next Two Lectures

Lecture 2: Lexical meaning in FGA

- **Modification**: *red ball*, *heavy idea*. What the scalar part of a product scores, and what the bivector part keeps.
- **Selection and coercion**: *begin the book*. An operation that changes the type of its argument.
- **Regular polysemy**: *the book is heavy* and *the book is boring*. One operation that applies across many words.

Lecture 3: Negation, connectives, quantifiers

- **Negation**: the complement, and why *not hot* is not *cold*; antonymy as a reflection.
- **Connectives**: *and* and *or* over subspaces.
- **Quantifiers and monotonicity**: *every*, *some*, *no*, and which inferences each one licenses.

Summary

1. **Every binding reads the outer product**. MAP takes its diagonal, HRR its fold, the dot product its trace; a bilinear form is a weighted reading of it.
2. **A commutative binding ignores the antisymmetric half**, and that half is where order lives. This is why MAP and HRR needed role vectors.
3. **The geometric product keeps the dot product and all of the antisymmetric half**: $\mathbf{uv} = \mathbf{u} \cdot \mathbf{v} + \mathbf{u} \wedge \mathbf{v}$, with order in the sign of the wedge.
4. **A unit vector is its own inverse**, as in MAP: $\mathbf{r}(\mathbf{rf}) = \mathbf{f}$ exactly.
5. **Crosstalk lands in another grade**. With orthogonal roles, superposed bindings decode exactly by contraction, which replaces the cleanup memory.
6. **A relation is a grade-2 object across two slots**: all of the bilinear form, applied one argument at a time, in either order.
7. **FGA reads grades as types and contraction as application**: *loves* is grade 2, *loves Mary* grade 1, *John loves Mary* a degree.

Exercises

1. Compute $\mathbf{uv}$ and $\mathbf{vu}$ for $\mathbf{u} = (1, 1, 0)$ and $\mathbf{v} = (0, 1, 1)$. Check that swapping the factors flips only the bivector part.
2. Show that MAP's binding is the diagonal of $\mathbf{uv}^\top$, and use the symmetric–antisymmetric split to prove that it is commutative.
3. Bind role $\mathbf{e}_1$ to filler $\mathbf{e}_3$ and role $\mathbf{e}_2$ to filler $\mathbf{e}_4$, and add the results. Unbind with $\mathbf{e}_2$ by multiplying and keeping the grade-1 part. What is the crosstalk term, and what is its grade?
4. *Marry* has $G = \left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$. Find its symmetric and antisymmetric parts. What happens if it is stored as a bivector over one entity space? Compute *John married Mary* and *Mary married John* with $\text{john} = (1, 0)$, $\text{mary} = (0.6, 0.8)$.
5. Derive *Mary loves John* with L_{love} from the slides, supplying the subject first. Give the object at each step and its grade.
6. $\star$ Draw roles and fillers as random unit vectors in $n = 200$ dimensions, so that they are only nearly orthogonal. Superpose three bindings, unbind each by contraction, and measure the cosine with the true filler. Compare with MAP at the same n.

Answers to Selected Exercises

1. $\mathbf{u}\cdot\mathbf{v}=1$. The planes: $\mathbf{e}_{12}$: $1\cdot 1-1\cdot 0=1$; $\mathbf{e}_{13}$: $1\cdot 1-0\cdot 0=1$; $\mathbf{e}_{23}$: $1\cdot 1-0\cdot 1=1$. So $\mathbf{uv}=1+\mathbf{e}_{12}+\mathbf{e}_{13}+\mathbf{e}_{23}$ and $\mathbf{vu}=1-\mathbf{e}_{12}-\mathbf{e}_{13}-\mathbf{e}_{23}$.

3. $\mathbf{E}=\mathbf{e}_1\mathbf{e}_3+\mathbf{e}_2\mathbf{e}_4=\mathbf{e}_{13}+\mathbf{e}_{24}$. Then $\mathbf{e}_2\mathbf{E}=\mathbf{e}_2\mathbf{e}_1\mathbf{e}_3+\mathbf{e}_2\mathbf{e}_2\mathbf{e}_4=-\mathbf{e}_{123}+\mathbf{e}_4$. The grade-1 part is $\mathbf{e}_4$, the filler, exactly. The crosstalk $-\mathbf{e}_{123}$ is grade 3.

4. $S=G$ and $A=0$: *marry* is symmetric. As a bivector over one entity space it would be 0: the whole relation vanishes. With G: *John married Mary* $=(1,0)\,G\,(0.6,0.8)^{\top}=0.8$, and *Mary married John* $=0.8$, as the entailment requires.

5. mary as subject is $0.6\,\mathbf{s}_1+0.8\,\mathbf{s}_2$ (grade 1). Contracting into L: $0.6(0.2\,\mathbf{o}_1+0.9\,\mathbf{o}_2)+0.8(0.3\,\mathbf{o}_1+0.1\,\mathbf{o}_2)=0.36\,\mathbf{o}_1+0.62\,\mathbf{o}_2$, the property *loved by Mary* (grade 1). Then with john $=\mathbf{o}_1$ as object: 0.36 (grade 0), the same as $(0.6,0.8)\,G\,(1,0)^{\top}$.

Reading

- P. Smolensky, Tensor product variable binding and the representation of symbolic structures in connectionist systems, *Artificial Intelligence* 46:159–216, 1990.
- T. A. Plate, *Holographic Reduced Representation: Distributed Representation for Cognitive Structures*, CSLI, 2003.
- R. W. Gayler, Vector symbolic architectures answer Jackendoff's challenges for cognitive neuroscience, *ICCS/ASCS Joint Conference*, 2003.
- P. Kanerva, Hyperdimensional computing, *Cognitive Computation* 1:139–159, 2009.
- D. Aerts, M. Czachor & B. De Moor, Geometric analogue of holographic reduced representation, *Journal of Mathematical Psychology* 53(5):389–398, 2009.
- L. Dorst, D. Fontijne & S. Mann, *Geometric Algebra for Computer Science*, Morgan Kaufmann, 2007. Chapters 2–3: the outer product and contraction.
- B. Coecke, M. Sadrzadeh & S. Clark, Mathematical foundations for a compositional distributional model of meaning, *Linguistic Analysis* 36:345–384, 2010.
- I. Heim & A. Kratzer, *Semantics in Generative Grammar*, Blackwell, 1998. Function application and types.
- J. Pustejovsky, Toward a functional geometric algebra for natural language semantics, arXiv 2604.25902, 2026.